

Technical Documentation

# Secure Digital Identity Verification System

End-to-end architecture, workflow, and code reference for the Laxmi Sunrise Bank portal and Nagarik identity app demo — HMAC-signed QR verification with SignalR auto-fill.

Project: DigitalIdentitySecureFill

Version: 1.0 | Date: June 2026

Stack: ASP.NET Core 8, React, PostgreSQL, Docker, SignalR

## Table of Contents

---

1. Executive Summary — What This Demo Does
2. System Architecture Overview
3. Deployment & Docker Topology
4. End-to-End Verification Workflow
5. Security Model
6. Database Design
7. Banking API — Endpoints & Code
8. Identity API — Endpoints & Code
9. Banking Portal Frontend (React)
10. Nagarik App Frontend (React)
11. Configuration & Environment Variables
12. Demo Credentials & How to Run
13. Project File Structure

## 1. Executive Summary — What This Demo Does

DigitalIdentitySecureFill is a production-style proof-of-concept that simulates how a bank (Laxmi Sunrise Bank) can verify a customer's identity through Nepal's national Nagarik digital identity app — without the customer manually retyping KYC data on a long form.

Business problem: Opening a bank account online requires name, national ID, address, phone, and email. Re-entering this data is slow and error-prone. This system lets the citizen approve sharing verified identity data from Nagarik, and the bank form auto-fills in real time.

### What the user sees (demo flow)

1. Customer visits the Bank Portal (port 3000) and opens the Account Opening page.
2. They click Verify with Nagarik. A QR code appears with a 2-minute countdown.
3. On their phone, they open the Nagarik App (port 3001), log in, and upload a screenshot of the QR.
4. A consent popup asks permission to share identity with Laxmi Sunrise Bank.
5. After approval, the bank form auto-fills with name, national ID, DOB, address, phone, and email.
6. Customer reviews and submits the application.

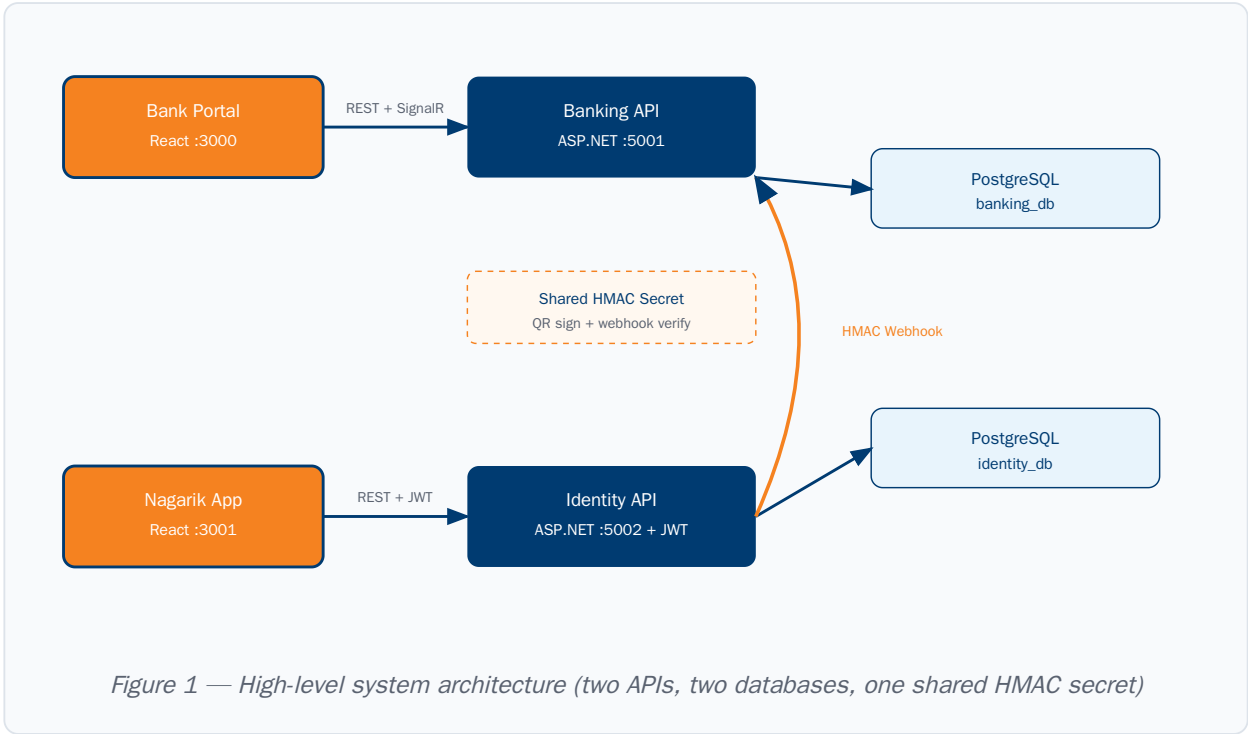
### What happens behind the scenes

- Bank API creates a unique requestId and signs the QR payload with HMAC-SHA256.
- Identity API validates the signature and expiry before showing consent.
- On consent, Identity API sends a signed webhook to the Banking API.
- Banking API marks the request complete and pushes data via SignalR to the browser.
- Polling (every 3 seconds) acts as a fallback if SignalR disconnects.

### Two independent systems

| System             | Role                                                       | Database    |
|--------------------|------------------------------------------------------------|-------------|
| Banking            | QR generation, webhook receiver, SignalR hub, account form | banking_db  |
| Identity (Nagarik) | Login, QR validation, consent, KYC profile, webhook sender | identity_db |

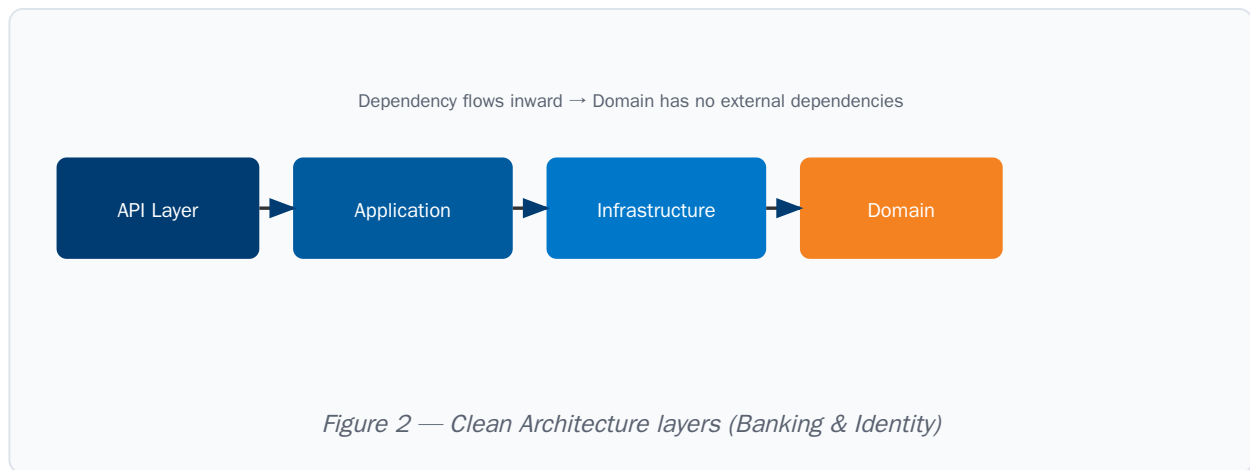
## 2. System Architecture Overview



### Clean Architecture (both backends)

Each .NET API follows the same layered pattern:

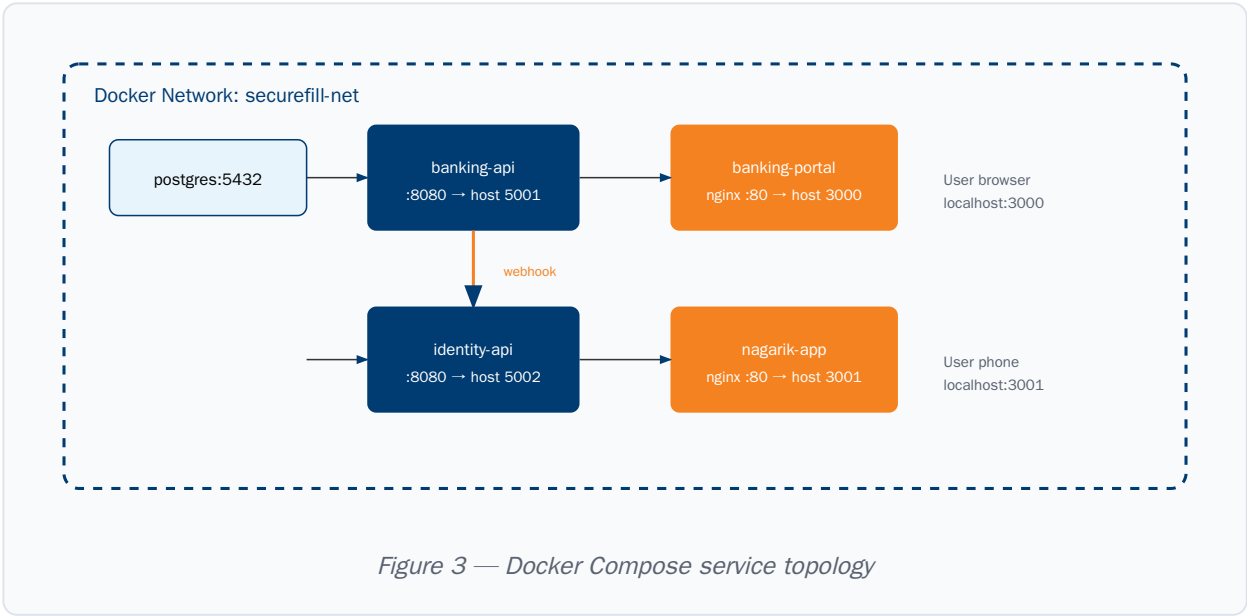
| Layer          | Responsibility                                        | Key technologies          |
|----------------|-------------------------------------------------------|---------------------------|
| Domain         | Entities, enums, value objects, domain events         | Pure C#                   |
| Application    | Commands, queries, DTOs, validators, interfaces       | MediatR, FluentValidation |
| Infrastructure | EF Core, repositories, HMAC, HTTP clients, migrations | PostgreSQL, Npgsql        |
| API            | Controllers, SignalR hubs, middleware, DI wiring      | ASP.NET Core 8            |



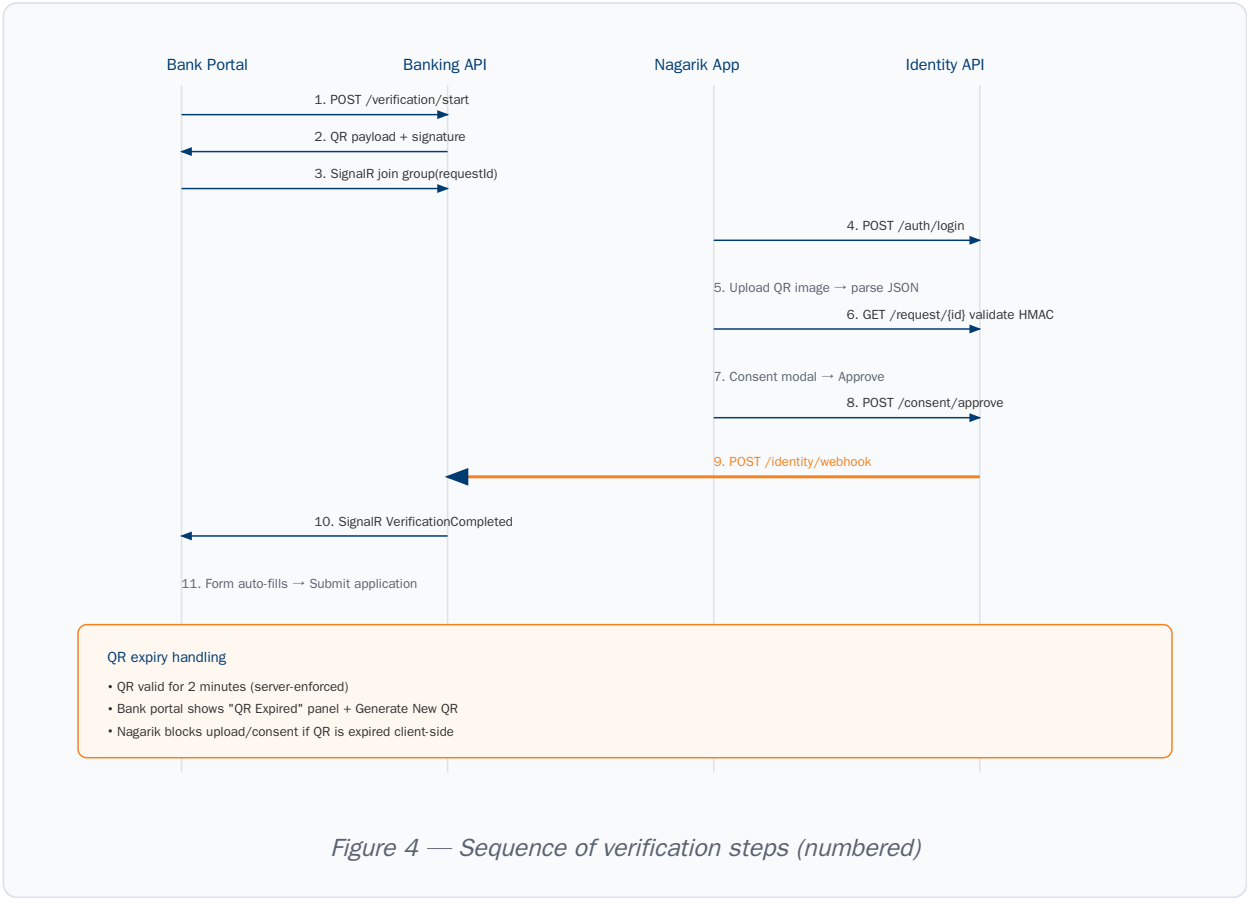
### 3. Deployment & Docker Topology

Run the full stack with `docker compose up --build` or `.\start-dev.ps1` on Windows.

| Service        | Container                 | Host Port   | Purpose                                                        |
|----------------|---------------------------|-------------|----------------------------------------------------------------|
| postgres       | securefill-postgres       | 5432        | PostgreSQL 16; creates banking_db + identity_db on first start |
| banking-api    | securefill-banking-api    | 5001 → 8080 | Banking REST API + SignalR hub                                 |
| identity-api   | securefill-identity-api   | 5002 → 8080 | Identity REST API + JWT auth                                   |
| banking-portal | securefill-banking-portal | 3000 → 80   | Nginx serves React; proxies /api and /hubs to banking-api      |
| nagarik-app    | securefill-nagarik-app    | 3001 → 80   | Nginx serves React; proxies /api to identity-api               |



4. End-to-End Verification Workflow



Verification request lifecycle (status enum)

| Status            | When                                                  |
|-------------------|-------------------------------------------------------|
| QrGenerated       | Bank starts verification; QR created                  |
| Pending / Scanned | Intermediate states (if used)                         |
| Completed         | Webhook received; identity data stored; SignalR fired |
| Expired           | Past expiry time without completion                   |
| Failed            | Validation or processing error                        |

## 5. Security Model

| Control           | Implementation                                                                                               |
|-------------------|--------------------------------------------------------------------------------------------------------------|
| QR integrity      | HMAC-SHA256 over {requestId} {bankId} {expiry:0} using shared secret. Identity API re-computes and compares. |
| Webhook integrity | HMAC-SHA256 over {requestId} {userId} {timestamp:0} {identityJson}. Banking API rejects invalid signatures.  |
| Time limit        | 2-minute QR expiry (UTC). Enforced on server and in both frontends.                                          |
| Replay prevention | IsUsed flag on verification request; unique ConsentRecord.RequestId in identity DB.                          |
| Authentication    | JWT on Identity API protected routes (login returns 1-hour token). Banking webhook uses HMAC, not JWT.       |
| Rate limiting     | AspNetCoreRateLimit on sensitive endpoints (start, login, consent, webhook).                                 |
| Audit trail       | AuditLog table in both databases records actions with IP and timestamps.                                     |

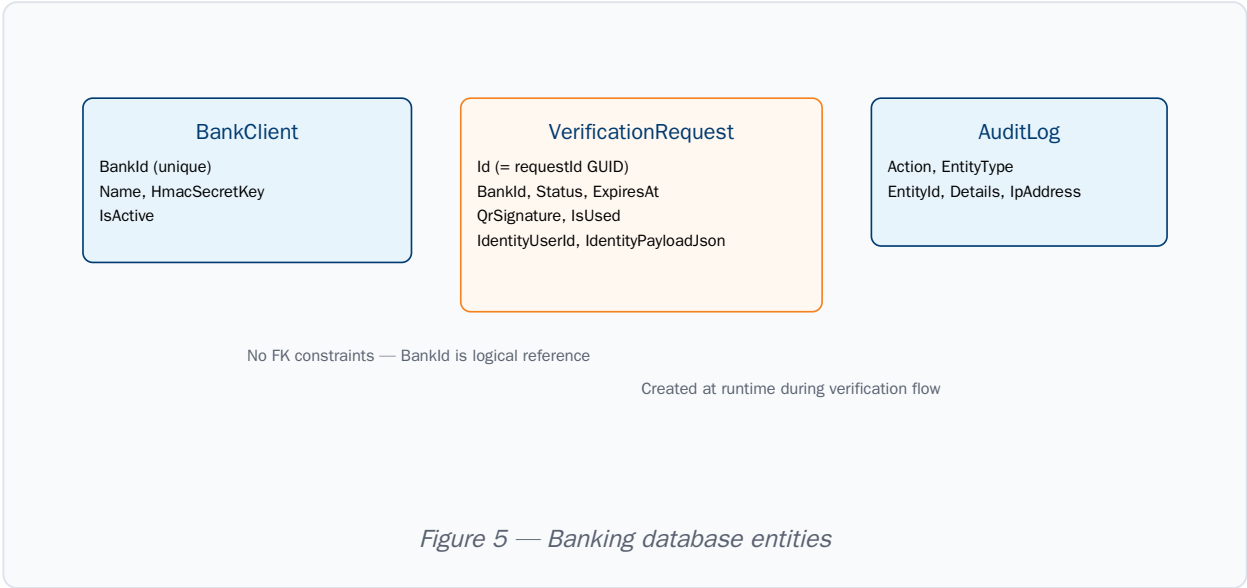
### QR payload JSON (embedded in QR code)

```
{
  "requestId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "bankId": "BANK-001",
  "expiry": "2026-06-12T10:02:00.0000000Z",
  "signature": "base64-encoded-hmac-sha256"
}
```

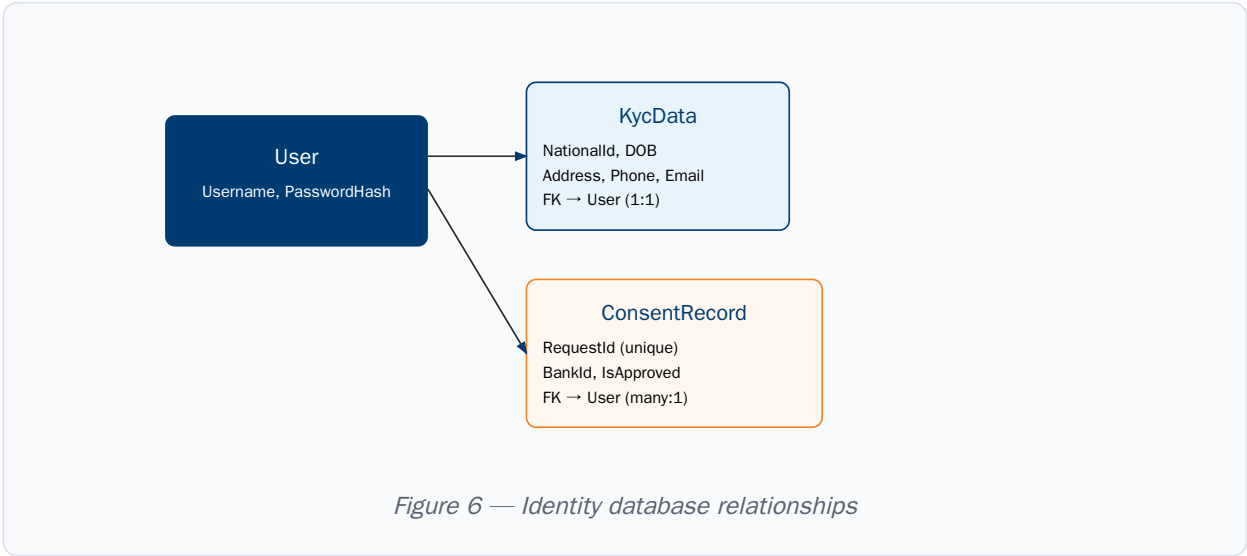


## 6. Database Design

### banking\_db



### identity\_db



### Seed data

| System  | Data                                               |
|---------|----------------------------------------------------|
| Banking | BANK-001 — Laxmi Sunrise Bank (active bank client) |

|          |                                                                                                            |
|----------|------------------------------------------------------------------------------------------------------------|
| Identity | <code>citizen / nagarik123</code> — Ram Bahadur Thapa<br><code>sita / nagarik123</code> — Sita Devi Sharma |
|----------|------------------------------------------------------------------------------------------------------------|

## 7. Banking API — Endpoints & Code

| Method | Route                         | Handler / Component                  | Purpose                                                 |
|--------|-------------------------------|--------------------------------------|---------------------------------------------------------|
| POST   | /api/verification/start       | StartVerificationCommandHandler      | Create request, sign QR, save to DB                     |
| GET    | /api/verification/{requestId} | GetVerificationQueryHandler          | Return status + identity data when complete             |
| POST   | /api/identity/webhook         | ProcessIdentityWebhookCommandHandler | Validate webhook HMAC, complete request, notify SignalR |
| WS     | /hubs/verification            | VerificationHub                      | Real-time push to bank portal                           |

### StartVerificationCommandHandler — what it does

1. Loads BankClient by bankId; fails if missing or inactive.
2. Generates new requestId (GUID) and expiry = UTC now + 2 minutes.
3. Builds canonical string requestId|bankId|expiry:0 and signs with HMAC.
4. Persists VerificationRequest with status QrGenerated.
5. Writes audit log VerificationStarted.
6. Returns JSON including qrPayload string for the QR code.

File:

banking/src/Banking.Application/Features/Commands/StartVerification/StartVerificationCommandHandler.cs

### ProcessIdentityWebhookCommandHandler — what it does

1. Loads verification request by requestId.
2. Verifies webhook HMAC signature matches payload.
3. Rejects if already used or expired.
4. Sets status Completed, stores identity JSON, IsUsed = true.

5. Calls `ISignalRVerificationNotifier` to broadcast `VerificationCompleted`.

### VerificationHub (SignalR)

- Clients call `JoinVerificationGroup(requestId)` after starting verification.
- Server sends `VerificationCompleted` with `identityData` to that group.
- File: `banking/src/Banking.API/Hubs/VerificationHub.cs`

## 8. Identity API — Endpoints & Code

| Method | Route                    | Auth | Handler                            | Purpose                                     |
|--------|--------------------------|------|------------------------------------|---------------------------------------------|
| POST   | /api/auth/login          | No   | LoginCommandHandler                | BCrypt password check; return JWT           |
| GET    | /api/request/{requestId} | JWT  | GetVerificationRequestQueryHandler | Validate QR HMAC, expiry, no prior consent  |
| POST   | /api/consent/approve     | JWT  | ApproveConsentCommandHandler       | Record consent; POST webhook to Banking API |
| GET    | /api/profile             | JWT  | GetUserProfileQueryHandler         | Return logged-in user's KYC                 |

### ApproveConsentCommandHandler — what it does

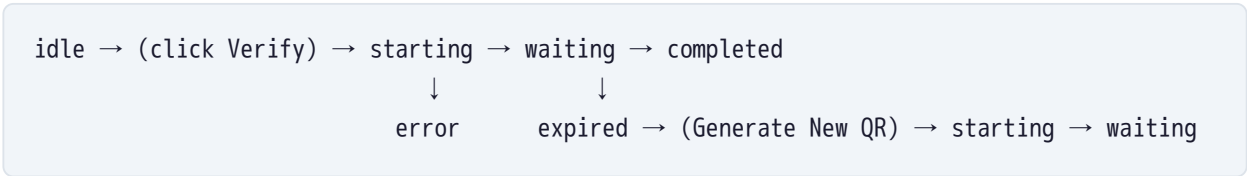
1. Re-validates QR parameters (HMAC, expiry).
2. Checks no existing ConsentRecord for this requestId (replay).
3. Creates consent record linked to authenticated user.
4. Loads user KYC data from KycData table.
5. Builds webhook body; signs with HMAC.
6. HTTP POST to BankingApi:BaseUrl/api/identity/webhook (Docker: http://banking-api:8080).

## 9. Banking Portal Frontend (React)

Location: banking/frontend/ — Vite + React Router, port 3000.

| File                       | What it does                                                                                                                     |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| pages/Home.jsx             | Landing page with Laxmi Sunrise branding and link to Open Account                                                                |
| pages/OpenAccount.jsx      | Main flow: start verification, show QR, status, and account form                                                                 |
| hooks/useVerification.js   | State machine: idle → starting → waiting → completed/expired/error. Manages SignalR + 3s polling. Detects QR expiry client-side. |
| components/QRCodeBox.jsx   | Renders QR via qrcode.react; countdown timer; expired panel with "Generate New QR"                                               |
| components/Form.jsx        | KYC fields (disabled until verification complete)                                                                                |
| components/StatusPanel.jsx | Shows current verification status and SignalR connection state                                                                   |
| services/api.js            | startVerification(), getVerificationStatus()                                                                                     |
| services/signalr.js        | Connects to /hubs/verification, joins group, listens for VerificationCompleted                                                   |
| utils/qrGenerator.js       | buildQrPayload(), isQrExpired(), getExpiryCountdown()                                                                            |

### useVerification state flow



## 10. Nagarik App Frontend (React)

Location: `identity/frontend/` — Vite + React Router, port 3001.

| File                                     | What it does                                                                                                      |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>pages/Scan.jsx</code>              | Login form + QR upload flow + consent modal orchestration                                                         |
| <code>pages/Profile.jsx</code>           | Displays KYC profile (requires login)                                                                             |
| <code>components/QRUploader.jsx</code>   | File upload; decodes QR image using <code>html5-qrcode</code>                                                     |
| <code>components/ConsentModal.jsx</code> | Popup asking user to approve sharing identity with bank                                                           |
| <code>hooks/useAuth.js</code>            | JWT stored in <code>localStorage</code> ( <code>nagarik_token</code> , <code>nagarik_user</code> )                |
| <code>utils/qrParser.js</code>           | <code>parseQrPayload()</code> , <code>isQrExpired()</code> — validates JSON structure                             |
| <code>services/api.js</code>             | <code>login()</code> , <code>validateRequest()</code> , <code>approveConsent()</code> , <code>getProfile()</code> |

### Scan.jsx flow

1. User logs in → JWT saved.
2. User uploads QR screenshot → parse JSON payload.
3. Client checks expiry → if expired, show error (no consent).
4. Call `GET /api/request/{id}` with signature query params.
5. Show `ConsentModal` on success.
6. On approve → `POST /api/consent/approve` → success banner.

## 11. Configuration & Environment Variables

---

### Banking API (appsettings / Docker env)

| Key                         | Purpose                 |
|-----------------------------|-------------------------|
| ConnectionStrings:BankingDb | PostgreSQL connection   |
| Security:HmacSecretKey      | Must match Identity API |
| Cors:AllowedOrigins         | http://localhost:3000   |

### Identity API

| Key                               | Purpose                                   |
|-----------------------------------|-------------------------------------------|
| ConnectionStrings:IdentityDb      | PostgreSQL connection                     |
| Security:HmacSecretKey            | Shared with Banking API                   |
| Jwt:Key, Jwt:Issuer, Jwt:Audience | JWT token signing                         |
| BankingApi:BaseUrl                | Webhook destination (internal Docker URL) |

### Frontend (Vite)

| Variable              | App         | Purpose                                  |
|-----------------------|-------------|------------------------------------------|
| VITE_BANKING_API_URL  | Bank portal | API base (empty = same-origin via nginx) |
| VITE_SIGNALR_URL      | Bank portal | SignalR hub path                         |
| VITE_NAGARIK_APP_URL  | Bank portal | For deep links (optional)                |
| VITE_IDENTITY_API_URL | Nagarik     | API base                                 |

## 12. Demo Credentials & How to Run

---

### URLs

| Service | URL |
|---------|-----|
|---------|-----|



|                  |                               |
|------------------|-------------------------------|
| Bank Portal      | http://localhost:3000         |
| Nagarik App      | http://localhost:3001         |
| Banking Swagger  | http://localhost:5001/swagger |
| Identity Swagger | http://localhost:5002/swagger |

## Credentials

| Use           | Value                                     |
|---------------|-------------------------------------------|
| Nagarik login | citizen / nagarik123 or sita / nagarik123 |
| Bank ID       | BANK-001                                  |

## Commands

```
# Full stack (recommended)
docker compose up --build

# Windows shortcut
.\start-dev.ps1

# Stop
docker compose down

# Regenerate this PDF
.\docs\generate-pdf.ps1
```

## 13. Project File Structure

---

```

DigitalIdentitySecureFill/
├── README.md
├── docker-compose.yml
├── docker-compose.supabase.yml    # Optional cloud DB overlay
├── start-dev.ps1
├── docs/
│   ├── SecureDigitalIdentityFill-Documentation.html
│   ├── SecureDigitalIdentityFill-Documentation.md
│   ├── SecureDigitalIdentityFill-Documentation.pdf
│   └── generate-pdf.ps1
├── docker/postgres/init/
│   └── 01-create-databases.sql
├── banking/
│   ├── Dockerfile
│   ├── frontend/                # React bank portal
│   └── src/
│       ├── Banking.Domain/
│       ├── Banking.Application/  # CQRS handlers
│       ├── Banking.Infrastructure/ # EF Core, HMAC
│       └── Banking.API/          # Controllers, SignalR
└── identity/
    ├── Dockerfile
    ├── frontend/                # React Nagarik app
    └── src/
        ├── Identity.Domain/
        ├── Identity.Application/
        ├── Identity.Infrastructure/
        └── Identity.API/

```

Disclaimer: This is a demonstration system. Change all secrets (HMAC key, JWT key, database passwords) before any production deployment. The Nagarik app is a mock — not connected to a real government identity system.